

Singleton Design Pattern

↳ a class that is restricted to have one object

didn't handle
concurrency
that well



① Normal Method:

- private constructor
- public static getInstance method.

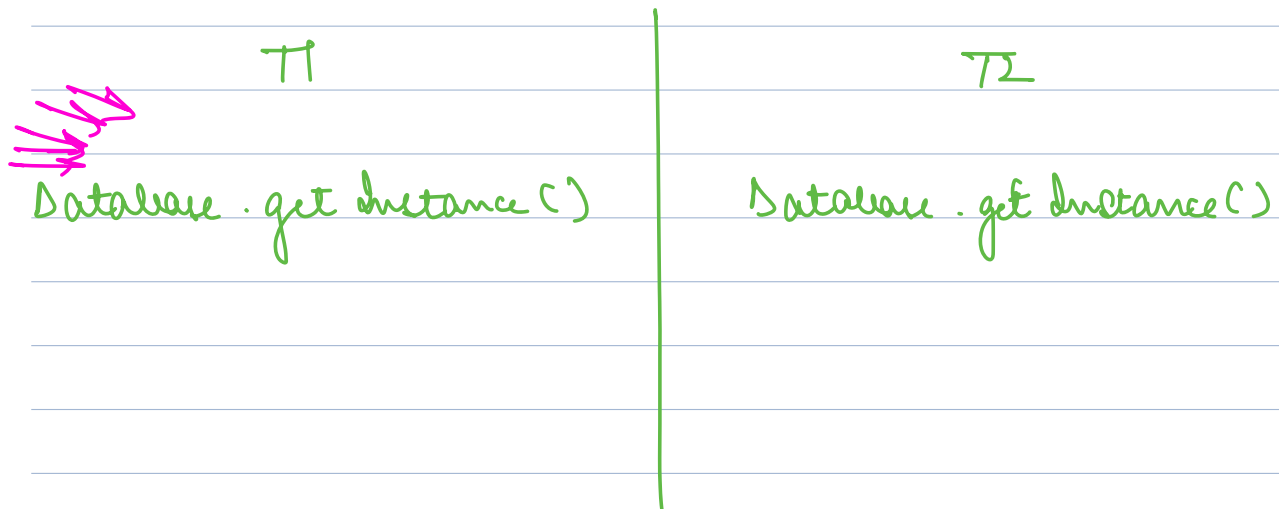
② Eager Initialization

- delay appⁿ startup time
- not always possible

③ Locks

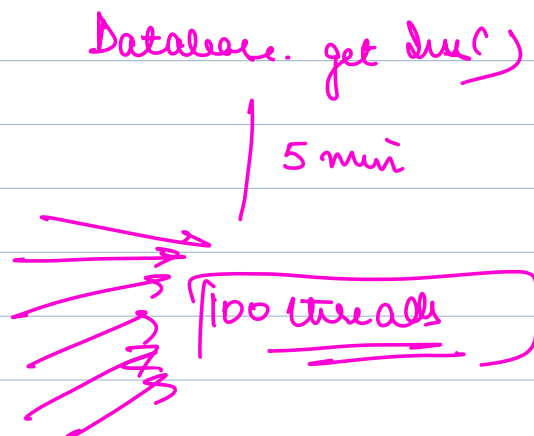
```
class Database {  
    =====  
    attr  
    =====  
}
```

```
private static Database instance;  
private Database() {}  
public synchronized static Database getInstance() {  
    T1 → T2 → if (instance == null)  
                instance = new Database();  
    T1 → T2 → return instance;  
}
```



→ Yay! Problem solved

→ AUT



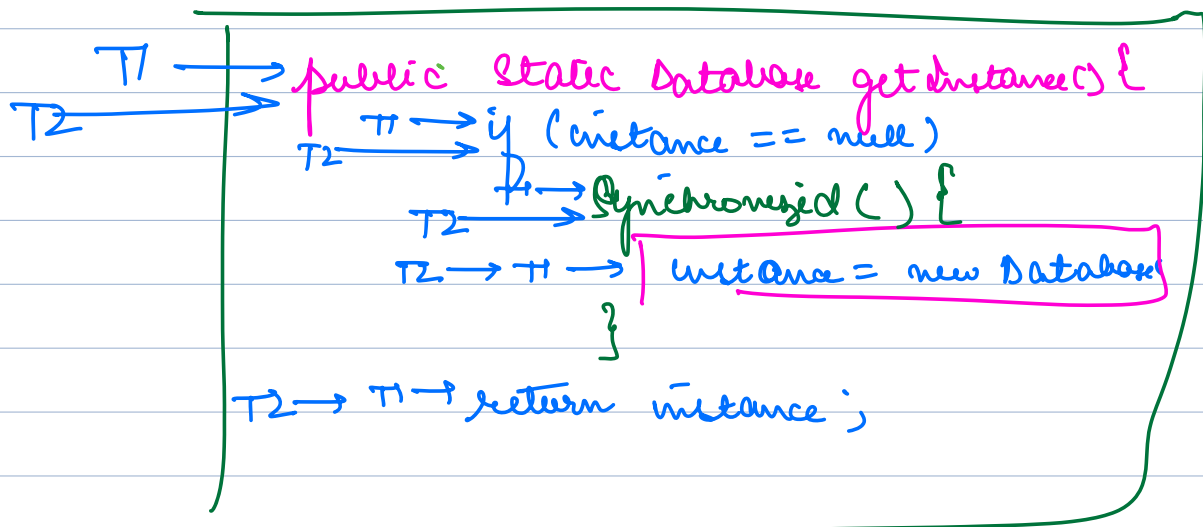
concurace

inital

→ Conc problem only existed during inital time of creating a DB. Once DB has been Created, conc is not an issue.



Almost
 → As every req will need a Database and as it will have to acquire lock ~~for~~ for every req, my appⁿ will be very slow!



If instance is already created, do we take any lock?

→ NO ⇒ No performance impact!!!

```

1
T1 → public static Database getInstance() {
T2 →   if (instance == null) {
T1 →     synchronized() {
T2 →       instance = new Database();
T1 →     }
T2 →   }
return instance;
}

```

```

T1 → public static Database getInstance() {
T2 →   if (instance == null) {
T2 →     synchronized() {
T1 →       if (instance == null)
T2 →         instance = new Database();
T2 →     }
T2 →   }
return instance;
}

```

this is called double check locking which is industry standard

DOUBLE CHECK LOCKING

- ① When lock is not needed it doesn't take any lock ⇒ Performance
- ② When a lock is needed it takes ⇒ Concurrency.

Dagger ⇒ Spring Boot

Still, Singleton can be broken in Java:

① Reflection: Adv Java

⇒ @ Getter

```
public class Student {  
    int age
```

```
    String name
```

```
    String gender.
```

```
    {  
        public int getAge() {  
            return age.  
        }  
    }
```

```
Student st = new Student().
```

```
st.getAge();
```

```
st.getName()
```

② Serialization

→ way to convert an object into
a stream of bytes.

Deserialization

→ way to convert series of bytes to
an object.

```
Animal {
    int age = 10
    String name = Naman
}
```

① Save obj on disk
→

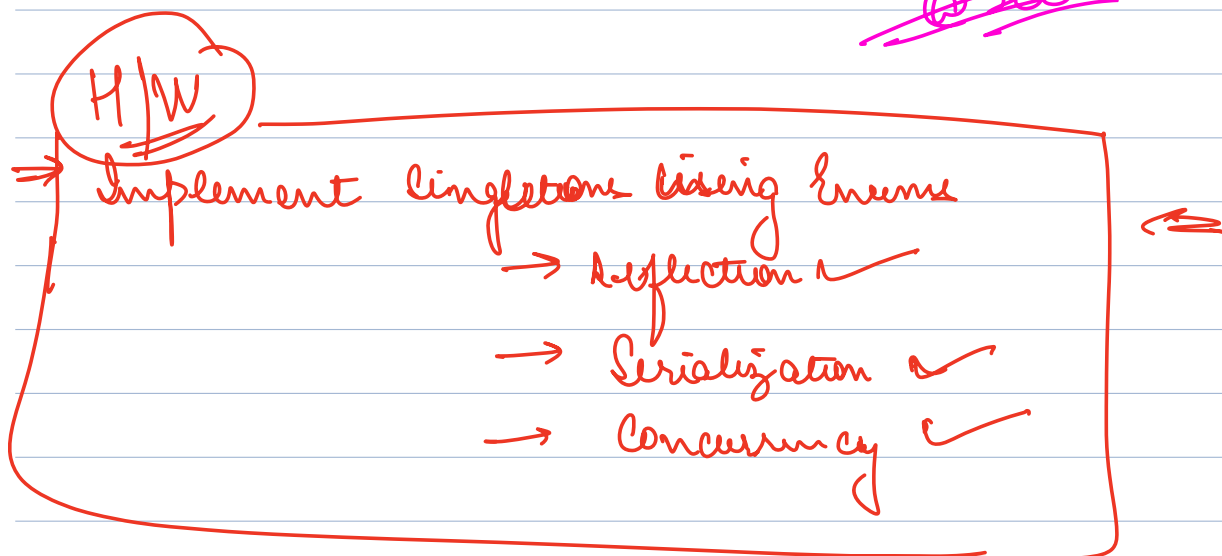
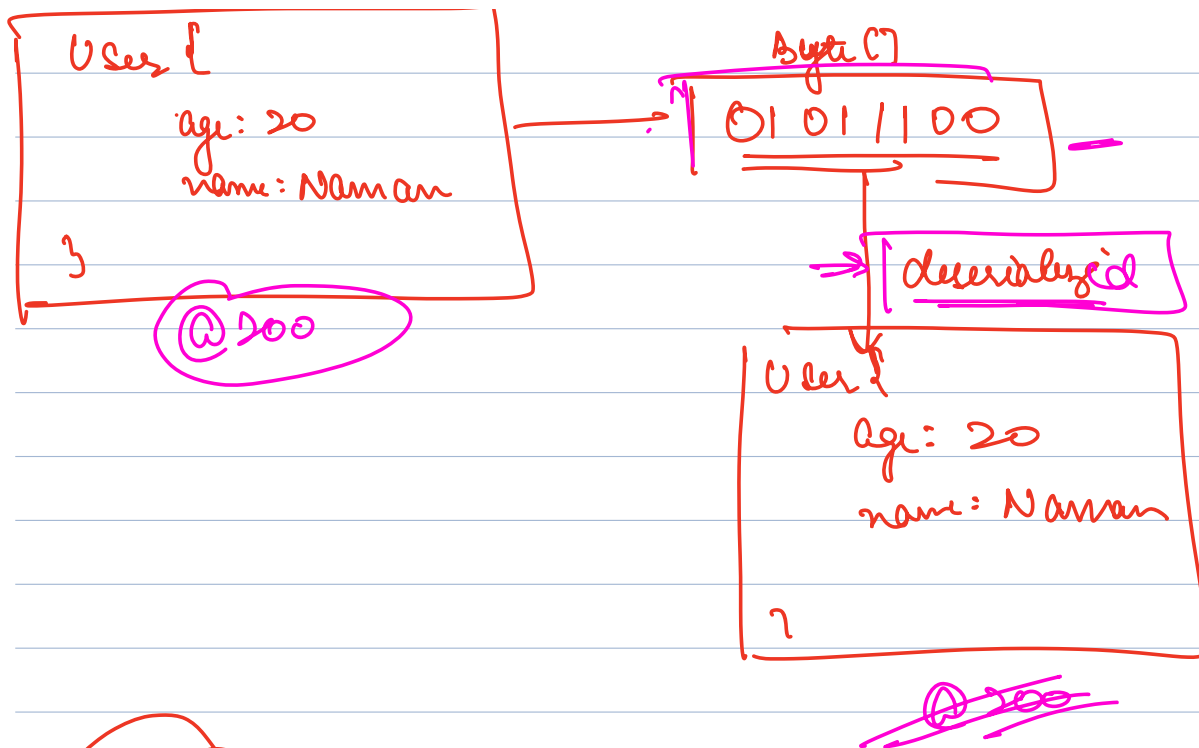
```
User Service {
    User login() {
    }
}
```

Client {

```
Database db = Database.getInstance()
```

```
byte[] o = Serializer.serialize(db)
```

```
Database db1 = Serializer.deserialize(o)
```



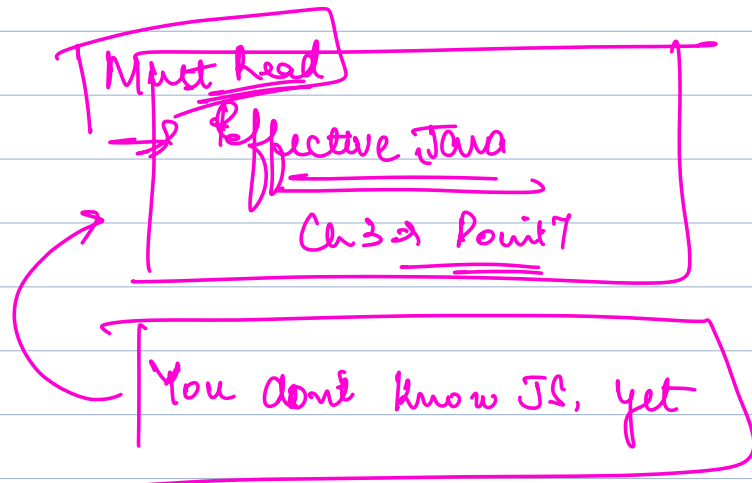
Revision

Singleton: Restricting class to 1 obj

How

- ① Private cons, static method
- ② eager initialization
- ③ Synchronized method
- ④ Double check
- ⑤ Enums

Hint
Strong Hint



Con

They make testing hard

① We need to test obj with diff obj.

②

flaky tests

→ Sometimes passing, sometimes failing
Solⁿ → Mock attributes of that class.

Cost they always respond to

what you want the way
you want

H/W

1.

How to create Singleton using Enum

Test cases $\Rightarrow$ reflection

OOP1

$\downarrow$

OOP4

Factory Design Pattern

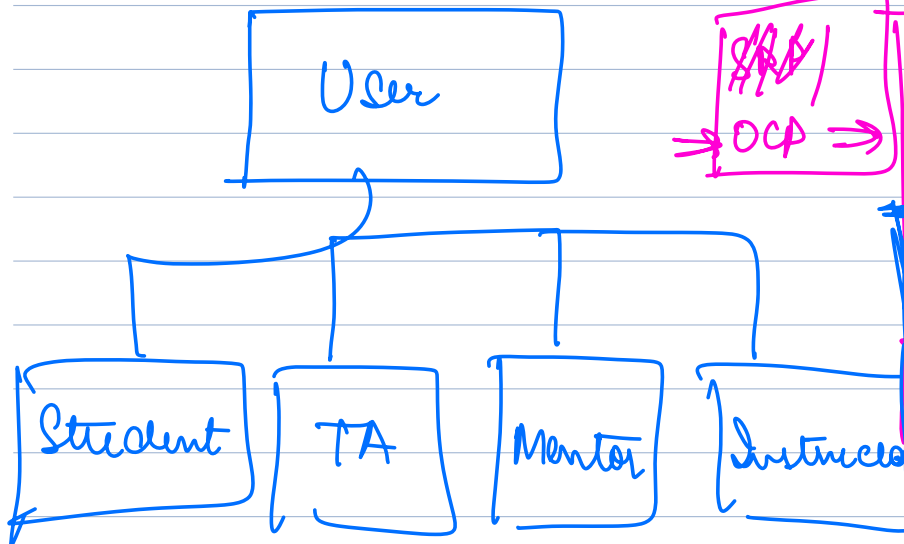


Manufacturer

→ Creational Design Pattern

Kurkure Factory

factory design pattern ⇒ { to help create an object of correct sub class of a parent class / interface }
⇒ create obj of correct type.



~~###~~
⇒ OCP ⇒

Menu Prog

```
String uip = ""
User u;

if (uip == "Student")
    u = new Student();
if (uip == "TA")
    u = new TA();
if (uip == "Mentor")
    u = new Mentor();
if (uip == "Instructor")
    u = new Instructor();
```

Creating an obj of correct type of
user based on input

Practical Factory Design Pattern: An object
~~of correct child class~~ based on
some condition

Prob Statement

- ① I have some condition
- ② I have some input
- ③ I need to create ^{new} object of corresponding child class
- ④ I might need to create that across multiple places in my codebase.

Conventional Solⁿ

wherever you need to create obj,
write if/else

Problems:

- ① SRP
- ② OCP
- ③ Code repetition

User v;
String vip = --;

if vip == --

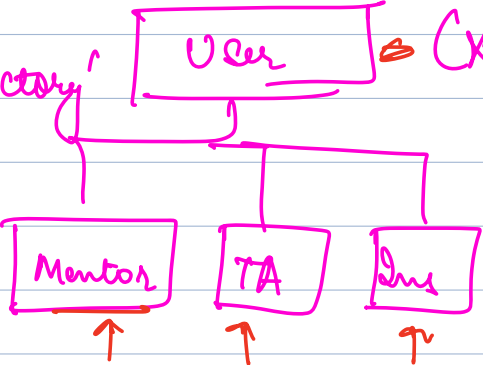
else vip == --

Note!

Think of practical design pattern whenever
you see a inheritance relⁿ

(parent - child, (interface imple))

IS-A

- (1) Create a class for 'X Factory' 
- (2) Create a method that takes input as its parameters

class User Factory {

static

User

get User for Input (String input) {

if (input == "mentor")

return new Mentor

if (input == 'ta')

return new TAC

if (input == 'Student')

return new Student()

~~DEPT SAP~~

→ no because
it is impossible
w/o if-else

}

}

User

get User for UserType (enum) { }

class Menu Program {

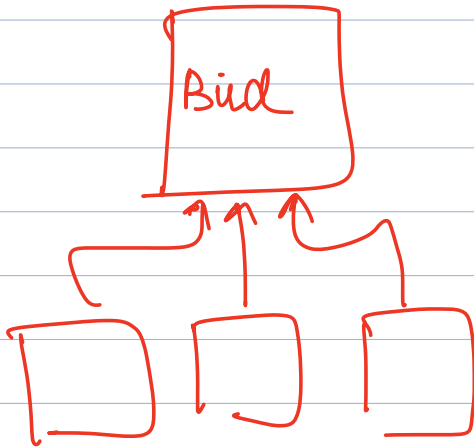
params {

String inp == ;

User u = User Factory . get User for Input
(inp);

}

Bird



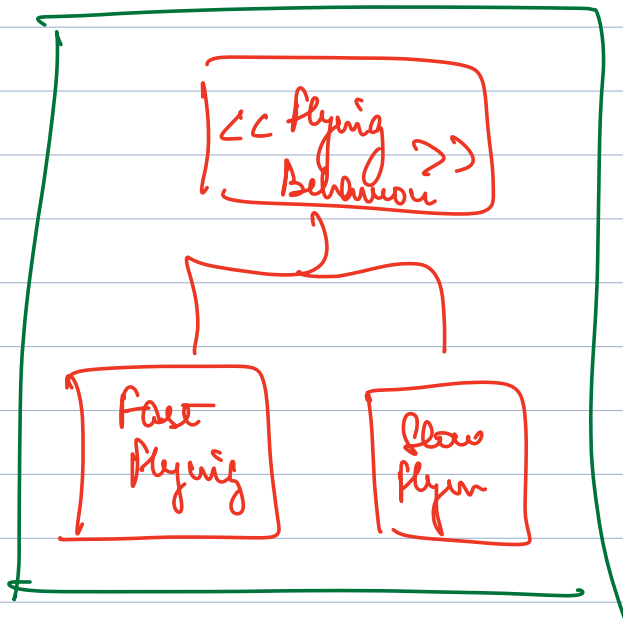
Bird Factory {

Bird getBirdPartType()

==

}

}



Plying Behaviour Factory {

Plying Behaviour
get Beh for Name(String)

==

}

}

Interface

List

l = new Array List<>()

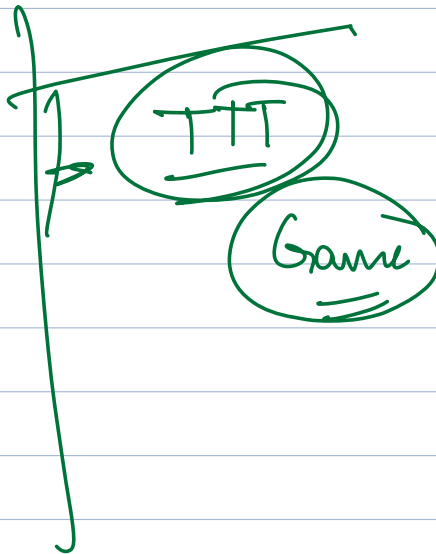
Client

→ get Product

String inp ⇒

return prod | based on inp.

ProductFactory {



}



Validation

⇒

Builder